

Protocol de Comunicare – PCD T17 Semantic Analyzer

Echipa proiect PCD

2026

1 Scopul documentului

Acest document descrie protocolul de comunicare folosit în cadrul proiectului PCD T17 – Semantic Analyzer for Source Code.

Protocolul este folosit pentru comunicarea dintre:

- serverul TCP;
- clientul normal în C;
- clientul admin în C;
- clientul alternativ în Python.

Aplicatia ruleaza local, folosind socket-uri INET peste TCP.

2 Informatii generale

Serverul asculta conexiuni pe:

Host: 127.0.0.1 Port: 8080 Protocol: TCP
--

Fiecare client se conecteaza la server, trimite o cerere, primeste raspunsul si apoi conexiunea este inchisa sau continuata, in functie de tipul de client si operatie.

Serverul proceseaza conexiunile in mod concurent, folosind procese copil pentru clienti.

3 Rolurile aplicatiei

3.1 Client normal

Clientul normal este folosit pentru:

- autentificare ca utilizator obisnuit;
- trimiterea unui fisier sursa catre server;
- primirea rezultatului analizei;
- descarcarea unui raport generat anterior.

3.2 Client admin

Clientul admin este folosit pentru:

- afisarea statisticilor serverului;
- afisarea logurilor;
- listarea fisierelor uploadate;
- listarea rapoartelor generate.

3.3 Client Python

Clientul Python ofera o varianta alternativa de client, implementata intr-un alt limbaj. Acesta poate trimite fisiere pentru analiza si poate descarca rapoarte.

4 Autentificare

La inceputul comunicarii, clientul trimite datele de autentificare catre server.

Credentialele sunt verificate folosind fisierul:

```
config/users.cfg
```

Un utilizator poate avea rol de:

- user;
- admin.

Serverul valideaza datele si permite accesul la operatii in functie de rol.

5 Model general de comunicare

Modelul general este:

```
Client -> Server: cerere  
Server -> Client: raspuns
```

Pentru operatiile care implica fisiere:

```
Client -> Server: metadata fisier  
Client -> Server: continut fisier  
Server -> Analyzer: fisier salvat  
Analyzer -> Server: raport generat  
Server -> Client: rezultat analiza
```

6 Operatii client normal

6.1 Upload fisier

Clientul trimite un fisier sursa catre server.

Exemplu comanda:

```
./client tests/sample.c
```

Pasi:

1. clientul se conecteaza la server;
2. clientul se autentifica;
3. clientul trimite numele fisierului;
4. clientul trimite dimensiunea fisierului;
5. clientul trimite continutul fisierului;
6. serverul salveaza fisierul in **uploads/**;
7. serverul ruleaza analyzer-ul;
8. serverul salveaza raportul in **reports/**;
9. serverul trimite rezultatul catre client.

6.2 Download raport

Clientul poate cere descarcarea unui raport deja generat.

Exemplu conceptual:

```
download sample_report.txt
```

Pasi:

1. clientul trimite numele raportului;
2. serverul cauta raportul in **reports/**;
3. serverul trimite continutul raportului;
4. clientul salveaza raportul in **downloads/**.

7 Operatii client Python

Clientul Python foloseste doua operatii principale.

7.1 Upload

```
python3 python_client/client.py upload tests/sample.c
```

Operatia trimite fisierul catre server si primeste rezultatul analizei.

7.2 Download

```
python3 python_client/client.py download sample_report.txt
```

Operatia descarca raportul din server si il salveaza in `downloads_py/`.

8 Operatii admin

Clientul admin trimite cereri speciale catre server. Aceste cereri sunt disponibile doar pentru utilizatorii cu rol de admin.

8.1 STATS

Cererea STATS returneaza informatii despre server.

Exemplu raspuns:

```
Server status: running
Analyzed files: 15
Last file: bad_type.c
```

8.2 LOGS

Cererea LOGS returneaza continutul fisierului de log:

```
logs/server.log
```

Exemplu de linii din log:

```
[2026-06-09 12:48:59] INFO File uploaded: sample.c
[2026-06-09 12:48:59] INFO Report saved for file: sample.c
[2026-06-09 12:48:59] INFO Analysis completed for file: sample.c
```

8.3 LIST UPLOADS

Aceasta cerere listeaza fisierele existente in:

```
uploads/
```

Exemplu raspuns:

```
Uploaded files:

* bad_type.c
* sample.c
```

8.4 LIST REPORTS

Aceasta cerere listeaza rapoartele existente in:

```
reports/
```

Exemplu raspuns:

Generated reports:

```
* sample_report.txt
* bad_type_report.txt
```

9 Formatul rapoartelor

Rapoartele generate de analyzer au format text.

Exemplu:

```
=== Analysis Result ===
Project: PCD T17 Semantic Analyzer
Source file: uploads/sample.c
Diagnostics count: 0
Functions count: 3
Variables count: 8
If statements count: 1
For loops count: 1
While loops count: 0

=== Diagnostics ===
Nu exista diagnostice.

Parse completed.
```

Pentru fisiere cu probleme, raportul contine diagnostice de tip warning sau error:

```
uploads/bad_type.c:10:9: error: incompatible pointer to integer conversion
```

10 Coduri de raspuns

Protocolul foloseste raspunsuri text simple. Cele mai importante categorii sunt:

Raspuns	Semnificatie
OK	operatia s-a realizat cu succes
ERROR	a aparut o eroare
AUTH OK	autentificare reusita
AUTH FAILED	autentificare esuata
BYE	conexiunea este inchisa

In implementare, raspunsurile pot include si text additional, in functie de operatie.

11 Gestionarea erorilor

Serverul trateaza urmatoarele situatii:

- fisier inexistent;

- utilizator invalid;
- rol insuficient pentru operatii admin;
- eroare la salvarea fisierului;
- eroare la rularea analyzer-ului;
- raport inexistent;
- conexiune inchisa de client.

Erorile importante sunt scrise in `logs/server.log`.

12 Concurenta

Serverul proceseaza clientii folosind procese copil. La fiecare conexiune acceptata, serverul creeaza un proces separat pentru client.

Flux general:

```
server accepta conexiune
server face fork()
procesul copil gestioneaza clientul
procesul parinte continua sa accepte alti clienti
```

Aceasta abordare permite procesarea mai multor clienti in paralel.

13 Exemplu test de concurenta

```
./client tests/sample.c & ./client tests/bad_type.c & wait
```

Daca serverul functioneaza corect, ambele fisiere sunt analizate si ambele rapoarte sunt generate.

14 Securitate si limitari

Protocolul este realizat pentru un proiect de laborator si ruleaza local. Autentificarea este simpla, pe baza fisierului `config/users.cfg`.

Limitari:

- nu exista criptare TLS;
- parolele nu sunt gestionate la nivel de productie;
- nu exista baza de date;
- comunicarea este gandita pentru rulare locala pe Linux/WSL.

15 Concluzie

Protocolul este simplu, text-based si potrivit pentru scopul proiectului. El permite autentificare, upload de fisiere, analiza semantica, generare de rapoarte, operatii de administrare si folosirea unui client alternativ in Python.